

hello.c

```
#include <stdio.h>

int main()
{
    printf("Hello World\n");
    return 0;
}
```

# Introduction to C-Programming

Concepts · Programs · Practical Solutions

## INSIDE THIS BOOK

- ✓ 6 units in easy story style, fully illustrated
- ✓ Previous year question paper with solutions
- ✓ Exam questions in every unit, plus mini projects
- ✓ 30 solved syllabus practicals
- ✓ Model question paper (new exam pattern)
- ✓ 130+ programs tested with GCC on Linux

## B.Sc. 1st Semester

Major & Minor · Computer Science

As per the FYUGP syllabus of Gauhati University under NEP 2020

AUTHOR

**Dr. Sisir Kumar  
Rajbongshi**

Department of Computer Science  
PDUAM, Amjonga, Goalpara

COM0100104MJ · COM0101004MN

ISBN 978-93-345-4161-8

ISBN allotted by Raja Rammohun Roy National Agency for ISBN, Department of Higher Education, Ministry of Education, Government of India

# Introduction to **C-Programming**

*Concepts · Programs · Practical Solutions*

As per the FYUGP Syllabus of Gauhati University under NEP 2020  
B.Sc. 1st Semester · Computer Science · Major and Minor  
COM0100104MJ · COM0101004MN

**Dr. Sisir Kumar Rajbongshi**

Assistant Professor, Department of Computer Science  
Pandit Deendayal Upadhyaya Adarsha Mahavidyalaya, Amjonga, Goalpara, Assam

## **Introduction to C-Programming**

As per the FYUGP Syllabus of Gauhati University under NEP 2020

B.Sc. 1st Semester, Computer Science (Major: COM0100104MJ; Minor: COM0101004MN)

**Author:** Dr. Sisir Kumar Rajbongshi

**First Edition:** 2026

**ISBN:** 978-93-345-4161-8

ISBN allotted by Raja Rammohun Roy National Agency for ISBN, Department of Higher Education, Ministry of Education, Government of India.

**Published by:** Dr. Sisir Kumar Rajbongshi (Author and Publisher)

Amjonga, Goalpara, Assam, India

E-mail: sisirrajbongshi@gmail.com

© Dr. Sisir Kumar Rajbongshi, 2026. All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior written permission of the author, except for brief quotations in reviews and for the personal, non-commercial study use of the programs.

**Disclaimer:** Every effort has been made to make this book accurate. All programs were compiled and run with GCC on Linux; results on other compilers or systems may differ slightly. The author shall not be responsible for any loss or damage arising from the use of the information in this book. Readers are requested to report errors to the e-mail address above.

Linux is a registered trademark of Linus Torvalds. Windows is a trademark of Microsoft Corporation. All other product names mentioned in this book are the property of their respective owners.

# Preface

---

The National Education Policy (NEP) 2020 has brought a major change to undergraduate education in India. Following this policy, Gauhati University introduced the Four-Year Undergraduate Programme (FYUGP), and in 2026 it adopted a revised syllabus for Computer Science. In the very first semester, students of both the Major and the Minor courses now study **Introduction to C-Programming** (COM0100104MJ / COM0101004MN). The course has six units, from the basics of C to file processing, and a practical component that the syllabus advises be carried out in a Linux environment.

While teaching this course to first-semester students at Pandit Deendayal Upadhyaya Adarsha Mahavidyalaya, Amjonga, I faced a simple problem: **to the best of my knowledge, no book follows this new syllabus unit by unit.** Students had to search through several books, each written for a different syllabus. Many of those books still use older practices such as `void main()`, `getch()` and `gets()`, which do not work on the Linux systems the syllabus recommends. Many students also come to this course without any background in computers, and for them a programming language can feel strange and difficult at first.

This book was written to meet these needs. It follows the Gauhati University FYUGP syllabus topic by topic, and it was designed around the questions students actually ask in class.

## Features of this book

- **Complete syllabus coverage.** All six units are covered in the order of the syllabus, with every topic explained in simple language.
- **Learning through stories.** Each topic begins with a short story from a first-semester classroom in Assam, so that a new idea feels familiar before it becomes technical. Small "Just for fun" boxes keep the learning light.
- **Programs that really run.** More than 130 programs are included, and every one of them was compiled and run with GCC on Linux. The output printed with each program is the actual output.
- **Standard, modern C.** The programs use standard C throughout, and the book points out practices that should be avoided.
- **Illustrations and flowcharts.** Diagrams show how memory, loops, arrays and file operations work.
- **Solved practicals.** Solutions are given for all 30 programs in the syllabus practical list.
- **Examination support.** Common mistakes, viva questions, exam questions and practice programs at the end of every unit; a previous year question paper with complete solutions; a model question paper on the new examination pattern; and guidance on planning answers in the examination hall.

I hope this book helps students not only to pass the examination but also to enjoy programming. Once you learn C well, languages such as C++, Java and Python become much easier to pick up.

No book is ever perfect. Suggestions from teachers and students for improving future editions will be gratefully received at [sisirrajbongshi@gmail.com](mailto:sisirrajbongshi@gmail.com).

**Dr. Sisir Kumar Rajbongshi**

Department of Computer Science  
Pandit Deendayal Upadhyaya Adarsha Mahavidyalaya  
Amjonga, Goalpara, Assam  
October 2026

# Acknowledgement

---

This book began in the classroom, and I owe it first to my students.

I am deeply thankful to the students of the Department of Computer Science, Pandit Deendayal Upadhyaya Adarsha Mahavidyalaya, Amjonga, especially those of the first FYUGP batches. Their honest questions, their difficulty in finding a book that followed the new syllabus, and their eagerness to learn showed me what was needed. Many times a student would stay after class and ask, "Sir, which book should I read for this topic?" More than anything else, that question led me to write this book. Every chapter tries to answer the doubts I heard in class, in the language in which they were asked.

I am grateful to my colleagues in the Department of Computer Science for their valuable suggestions and constant cooperation.

I remember with gratitude my teachers of the Department of Computer Science, Pragjyotish College, Guwahati, and of Gauhati University, from whom I first learned to love computer science.

Finally, I thank my family for their patience, support and understanding during the many hours this book took to complete.

**Dr. Sisir Kumar Rajbongshi**

# How to Use This Book

The book follows the six units of the syllabus in order, followed by the solved practicals, the question papers and the references.

| Feature            | What it is                                                                               |
|--------------------|------------------------------------------------------------------------------------------|
| Story              | A short scene from a first-semester class that introduces each topic                     |
| Definition         | The exact definition to learn for examinations                                           |
| Program and Output | A complete program and the output it actually produced                                   |
| Just for fun       | A light joke that helps you remember a point (Unit 3)                                    |
| Warning / Remember | Common traps and important rules                                                         |
| Review             | Common mistakes, viva and exam questions, and practice programs at the end of every unit |

- All programs follow standard C (`int main()` with `return 0;`) and were compiled and run with GCC on Linux, the environment advised in the syllabus. They also run in Code::Blocks, Dev-C++ and most online compilers.
- To compile and run a program on Linux: `gcc prog.c -o prog` and then `./prog`. Programs using `math.h` need `-lm` at the end.
- If you use Turbo C, keep all variable declarations at the top of each block, as the programs in this book do. A few C99 features are marked where they appear.
- Memory addresses shown in outputs (such as `0x7ffe5119ea8c`) will be different on every computer and every run.
- Write and run every program yourself. Then change it a little and see what happens. That is the fastest way to learn programming.

*The classroom stories in this book are imaginary. The names of the students are used only for illustration.*

# Contents

---

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Unit 1 · Getting Started with C Programming</b>       | <b>1</b>  |
| <b>Prologue: The First Class</b>                         | 2         |
| <b>Part A: The World of Programming</b>                  | 3         |
| 1. What Is a Program? What Is Programming? What Is Code? | 3         |
| 2. Programs All Around You                               | 4         |
| 3. Popular Programming Languages Today                   | 5         |
| 4. Why Learn C? A Short History                          | 6         |
| <b>Part B: Talking to the Computer</b>                   | 7         |
| 5. Low-Level vs High-Level Languages                     | 7         |
| 6. Compiled vs Interpreted Languages                     | 8         |
| 7. Structure of a C Program                              | 9         |
| 8. Header Files                                          | 10        |
| 9. The main() Function and Your First Program            | 11        |
| 10. Compiling and Executing a Program                    | 12        |
| <b>Part C: The Building Blocks of C</b>                  | 14        |
| 11. C Tokens                                             | 14        |
| 12. Statements and Expressions                           | 15        |
| 13. Data Types, Sizes and Ranges                         | 15        |
| 14. Variables                                            | 17        |
| 15. Constants                                            | 19        |
| <b>Part D: Operators</b>                                 | 21        |
| 16. Unary and Binary Operators                           | 21        |
| 17. Arithmetic Operators                                 | 21        |
| 18. Assignment Operators                                 | 22        |
| 19. Increment and Decrement Operators                    | 23        |
| 20. Comparison (Relational) Operators                    | 24        |
| 21. Logical Operators                                    | 25        |
| 22. Bitwise Operators                                    | 26        |
| 23. Conditional Operator                                 | 27        |
| 24. Precedence and Associativity                         | 28        |
| <b>Part E: Input, Output and Typecasting</b>             | 31        |
| 25. Character Input and Output                           | 31        |
| 26. puts() and printf()                                  | 33        |
| 27. Format Specifiers and Escape Sequences               | 34        |
| 28. scanf()                                              | 35        |
| 29. Typecasting                                          | 36        |
| 30. Mini Project: The PDUAM Canteen Bill                 | 37        |
| <b>Part F: Review</b>                                    | 39        |
| 31. Common Mistakes                                      | 39        |
| 32. Viva and Exam Questions                              | 39        |
| 33. Practice Programs                                    | 40        |
| <b>Unit 2 · Control Structures in C</b>                  | <b>42</b> |
| <b>Prologue: Programs That Think</b>                     | 43        |
| <b>Part A: Control Structures</b>                        | 44        |
| 1. What Is a Control Structure?                          | 44        |
| 2. Sequence, Selection and Iteration                     | 44        |

|                                           |           |
|-------------------------------------------|-----------|
| <b>Part B: Conditional Statements</b>     | <b>46</b> |
| 3. Simple if                              | 46        |
| 4. if-else                                | 47        |
| 5. else-if Ladder                         | 48        |
| 6. Nested if-else                         | 49        |
| 7. switch-case                            | 50        |
| <b>Part C: Loops</b>                      | <b>54</b> |
| 8. Why Loops?                             | 54        |
| 9. while Loop                             | 55        |
| 10. do-while Loop                         | 55        |
| 11. for Loop                              | 56        |
| 12. Counting Loops and Indefinite Loops   | 57        |
| 13. Nested Loops                          | 59        |
| 14. Comparing the Three Loops             | 61        |
| <b>Part D: Jump and Other Statements</b>  | <b>62</b> |
| 15. break                                 | 62        |
| 16. continue                              | 64        |
| 17. goto                                  | 65        |
| 18. return                                | 66        |
| 19. Null Statement and Block Statement    | 67        |
| <b>Part E: Mini Project: An ATM Menu</b>  | <b>69</b> |
| <b>Part F: Review</b>                     | <b>72</b> |
| 20. Common Mistakes                       | 72        |
| 21. Viva and Exam Questions               | 72        |
| 22. Practice Programs                     | 73        |
| <b>Unit 3 · Arrays and Strings</b>        | <b>75</b> |
| <b>Prologue: Sixty Variables</b>          | <b>76</b> |
| <b>Part A: One-Dimensional Arrays</b>     | <b>77</b> |
| 1. Why Arrays?                            | 77        |
| 2. Declaring an Array                     | 77        |
| 3. Initializing an Array                  | 78        |
| 4. Accessing Array Elements               | 79        |
| <b>Part B: Basic Operations on Arrays</b> | <b>81</b> |
| 5. Traversal                              | 81        |
| 6. Insertion, Deletion and Update         | 82        |
| 7. Searching                              | 83        |
| 8. Sorting                                | 86        |
| <b>Part C: Multidimensional Arrays</b>    | <b>88</b> |
| 9. Two-Dimensional Arrays                 | 88        |
| 10. Initializing and Reading 2-D Arrays   | 89        |
| 11. Matrix Programs                       | 90        |
| 12. Arrays with More Dimensions           | 93        |
| <b>Part D: Strings</b>                    | <b>94</b> |
| 13. Introduction to Strings               | 94        |
| 14. Declaring and Initializing Strings    | 94        |
| 15. String Input and Output               | 96        |
| 16. Working with Strings                  | 97        |
| 17. Array of Strings                      | 99        |
| 18. A Look at string.h                    | 99        |

|                                                          |            |
|----------------------------------------------------------|------------|
| <b>Part E: Mini Project: Class Result Analyser</b>       | <b>101</b> |
| <b>Part F: Review</b>                                    | <b>103</b> |
| 19. Common Mistakes                                      | 103        |
| 20. Viva and Exam Questions                              | 103        |
| 21. Practice Programs                                    | 104        |
| <b>Unit 4 · Functions and Pointers</b>                   | <b>106</b> |
| <b>Part A: Pointers</b>                                  | <b>107</b> |
| 1. Memory and Addresses                                  | 107        |
| 2. Introduction to Pointers                              | 107        |
| 3. Pointer Declaration and Initialization                | 107        |
| 4. Pointers and Addresses: the & and * Operators         | 108        |
| 5. Chain of Pointers (Pointer to Pointer)                | 110        |
| 6. Operations on Pointers (Pointer Arithmetic)           | 111        |
| 7. Pointers and Arrays                                   | 112        |
| 8. Dynamic Memory Allocation                             | 114        |
| <b>Part B: Functions</b>                                 | <b>118</b> |
| 9. Introduction to Functions                             | 118        |
| 10. Elements of a Function                               | 118        |
| 11. Function Arguments                                   | 119        |
| 12. Return Types and Categories of Functions             | 120        |
| 13. Function Calling: Call by Value vs Call by Reference | 123        |
| 14. Passing an Array as Argument to a Function           | 125        |
| 15. Basic Concept of Recursion                           | 127        |
| <b>Part C: Review</b>                                    | <b>131</b> |
| 16. Common Mistakes                                      | 131        |
| 17. Viva and Exam Questions                              | 131        |
| 18. Practice Programs                                    | 132        |
| <b>Unit 5 · Structures and Unions</b>                    | <b>133</b> |
| <b>Part A: Structures</b>                                | <b>134</b> |
| 1. Introduction: Why Structures?                         | 134        |
| 2. Structure Declaration                                 | 134        |
| 3. Declaring Structure Variables                         | 135        |
| 4. Structure Initialization                              | 136        |
| 5. Accessing Structure Members                           | 136        |
| 6. Operations on Structures                              | 138        |
| 7. Memory and Size of a Structure                        | 139        |
| 8. Array of Structures                                   | 140        |
| 9. Some Related Concepts (Enrichment)                    | 142        |
| <b>Part B: Unions</b>                                    | <b>147</b> |
| 10. Introduction to Unions                               | 147        |
| 11. Union Declaration and Initialization                 | 147        |
| 12. Accessing Union Members                              | 148        |
| 13. Where Unions Are Useful                              | 149        |
| <b>Part C: Comparison and Review</b>                     | <b>150</b> |
| 14. Difference between Structures and Unions             | 150        |
| 15. Structure vs Array                                   | 152        |
| 16. Common Mistakes                                      | 153        |
| 17. Viva and Exam Questions                              | 153        |
| 18. Practice Programs                                    | 154        |

|                                                              |            |
|--------------------------------------------------------------|------------|
| <b>Unit 6 · File Processing in C</b>                         | <b>155</b> |
| <b>Part A: Basics of File Handling</b>                       | 156        |
| 1. Introduction: Why Files?                                  | 156        |
| 2. Steps in File Handling and the FILE Pointer               | 156        |
| 3. Opening a File: fopen()                                   | 157        |
| 4. Closing a File: fclose()                                  | 158        |
| 5. Text Files vs Binary Files                                | 158        |
| <b>Part B: Reading and Writing Files</b>                     | 160        |
| 6. File I/O Functions at a Glance                            | 160        |
| 7. Writing with fprintf()                                    | 160        |
| 8. Reading Lines with fgets()                                | 160        |
| 9. Reading Formatted Data with fscanf()                      | 163        |
| 10. Character I/O: fgetc() and fputc()                       | 164        |
| 11. End of File: EOF and feof()                              | 166        |
| 12. Binary I/O: fwrite() and fread()                         | 167        |
| <b>Part C: Random Access to Files</b>                        | 169        |
| 13. Sequential vs Random Access                              | 169        |
| 14. ftell(), fseek() and rewind()                            | 169        |
| <b>Part D: Review</b>                                        | 174        |
| 15. Common Mistakes                                          | 174        |
| 16. Viva and Exam Questions                                  | 174        |
| 17. Practice Programs                                        | 175        |
| <b>Practicals · Solutions to the Syllabus Practical List</b> | <b>177</b> |
| <b>List of Practicals</b>                                    | 178        |
| Unit 1: Getting Started with C                               | 179        |
| Unit 2: Control Structures                                   | 180        |
| Unit 3: Arrays and Strings                                   | 190        |
| Unit 4: Functions and Pointers                               | 194        |
| Unit 5: Structures and Unions                                | 199        |
| Unit 6: File Processing                                      | 201        |
| <b>Appendix · Question Papers and Solutions</b>              | <b>204</b> |
| <b>Appendix A: The Examination</b>                           | 205        |
| A.1 Pattern of the question paper                            | 205        |
| A.2 Planning the two hours                                   | 205        |
| A.3 How much to write                                        | 205        |
| A.4 Frequently asked topics                                  | 206        |
| <b>Appendix B: Previous Year Question Paper, 2025</b>        | 207        |
| <b>Appendix C: Solutions to the 2025 Question Paper</b>      | 209        |
| Question 1: Objective ( $1 \times 5 = 5$ )                   | 209        |
| Question 2: Definitions ( $2 \times 5 = 10$ )                | 209        |
| Question 3: Short Answers ( $5 \times 4 = 20$ )              | 211        |
| Question 4: Long Answers ( $10 \times 1 = 10$ )              | 215        |
| <b>Appendix D: Model Question Paper</b>                      | 220        |
| D.2 Answers and Hints                                        | 221        |
| <b>References and Further Reading</b>                        | <b>223</b> |

## UNIT 1

# Getting Started with C Programming

---

**Syllabus (10 Lectures):** Introduction to programming languages - High-level vs low level languages, compiled vs interpreted languages. Structure of a C program. Introduction to Header files. Main function and a simple program execution. Compiling and executing a program. C tokens - keywords, identifiers, constants, operators. Statements and expressions in C. Basic data types in C - integers, floats, doubles, characters. Void. Size and range of values of data types. Variables. Constants - integer constant, real constant, character constant, string constant. Declaration and initialization of variables and constants. Assigning values to variables. Operators in C - binary and unary operators. Arithmetic, assignment, logical, comparison, bitwise and conditional operators. Order of precedence of operators. Associativity of operators. Input and output statements - getchar(), getc(), getch(), putchar(), putc(), puts(), scanf(), printf(), format specifiers. Typecasting.

## Prologue: The First Class



### STORY · MONDAY, 10 A.M.

*It was the first class of B.Sc. 1st Semester in the Department of Computer Science, PDUAM Amjonga. Kallal Kalita took a seat near the window. Sanjita Rabha was already writing the date in a new notebook. Arpan Jyoti Daimary and Sonajit Rabha were busy talking about a new mobile game.*

*Sir walked in, wrote one line on the board, and turned around.*

```
printf("Hello, World!");
```

*"By the end of today," he said, "each of you will make a computer say this. But first, a question. How many of you have used a computer program today?"*

*Only Arpan raised his hand. Sir smiled. "Kallal, how did you wake up this morning?"*

*"My phone alarm, Sir." "That alarm is a program. Sonajit, how did you pay for your*

*tea at the gate?" "UPI, Sir." "A program. Sanjita, how did you know the bus time?"*

*"WhatsApp group, Sir." "Program. You have all used dozens of programs before ten o'clock. This semester, you will learn to write one."*

# Part A: The World of Programming

## 1. What Is a Program? What Is Programming? What Is Code?

### STORY · A RECIPE FOR ASSAM TEA

"A computer is very fast," Sir said, "but it is not clever. It does exactly what it is told, step by step. Tell me how to make a cup of Assam tea." Sanjita listed the steps. Sonajit added, "Drink it!" Everyone laughed, but Sir wrote every step on the board.

### A RECIPE IS A PROGRAM FOR A PERSON

1. Boil one cup of water.
2. Add one spoon of tea leaves.
3. Boil for 2 minutes.
4. Add milk and sugar.
5. Boil for 1 more minute.
6. Strain into a cup.
7. Serve.

A recipe has a fixed list of steps in the right order. If a step is missing or in the wrong order, the tea is spoiled. A computer program works the same way.

### Definitions

**Program:** a set of instructions, written in a programming language, that tells a computer what to do and in what order, to solve a problem or complete a task.

**Programming:** the process of designing, writing, testing and correcting programs.

**Code (source code):** the text of a program as written by a programmer. Writing it is called *coding*.

**Programmer:** a person who writes programs.

**Algorithm:** the step-by-step method for solving a problem, before it is written in any language. The tea recipe above is an algorithm.

Every program, from a calculator to a space mission, does three basic things:

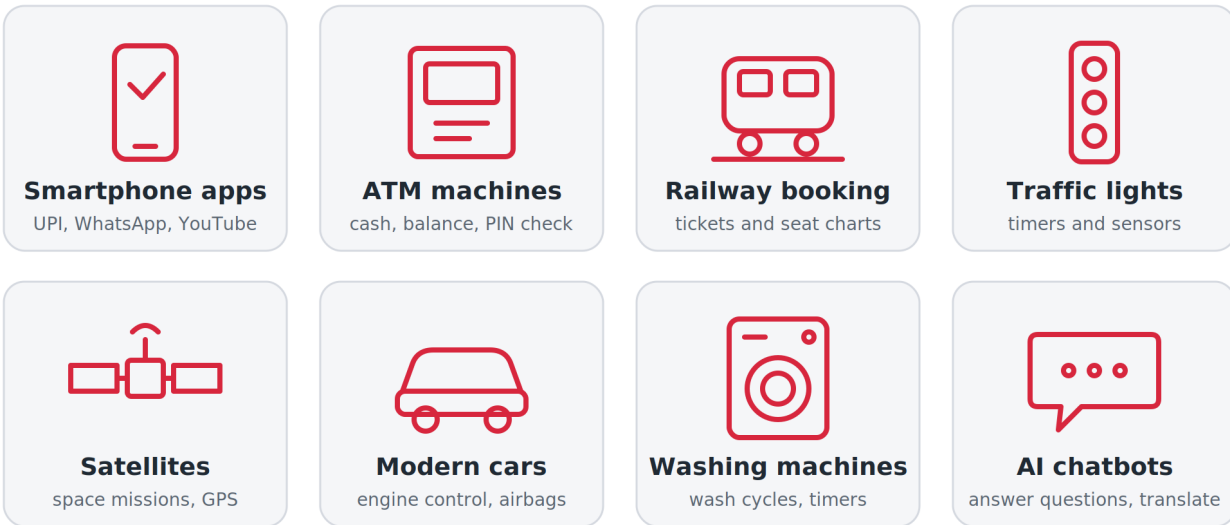
| Step    | Meaning          | Tea example             | Program example               |
|---------|------------------|-------------------------|-------------------------------|
| Input   | Take data in     | water, tea leaves, milk | two numbers from the keyboard |
| Process | Work on the data | boil, mix, strain       | add the numbers               |
| Output  | Give the result  | a cup of tea            | show the sum on the screen    |

### Did you know?

The word *computer* once meant a person who did calculations by hand. Today, the machine does the calculating, but a person still has to write the program that tells it how.

## 2. Programs All Around You

We usually think of a program as something on a laptop. In fact, programs are inside almost every modern machine. Here are a few you meet every day:



*Every one of these runs on programs written by programmers.*

- **On your phone:** UPI payments, WhatsApp, YouTube, Google Maps, online classes and mobile games are all programs (apps).
- **In banks and offices:** ATMs check your PIN and balance; railway and bus booking websites find seats and print tickets.
- **In the street:** traffic lights change on timers; CCTV cameras record; electricity meters count units.
- **At home:** washing machines, microwave ovens, smart TVs and air conditioners have small built-in computers running programs. These are called **embedded systems**.
- **In vehicles:** modern cars use programs to control fuel, brakes and airbags.
- **In space:** satellites for weather, TV and GPS, and space missions such as ISRO's Chandrayaan-3, which landed on the Moon in 2023, depend on programs.
- **Artificial Intelligence:** chatbots, voice assistants, translation apps and face unlock are programs that learn from data.

### Today's world of programming

Programming is no longer only for computer scientists. Doctors use programs to read scans, farmers use apps for weather and crop prices, and shopkeepers use them for billing. Programming skills are useful in almost every job. With programming, a student in Amjonga can build an app that is used in Assam, India, or anywhere in the world.

Some areas where programmers work today:

| Area                                     | What is built                                |
|------------------------------------------|----------------------------------------------|
| Mobile app development                   | Android and iPhone apps                      |
| Web development                          | Websites and online services                 |
| Artificial Intelligence and Data Science | Chatbots, predictions, recommendations       |
| Embedded systems and IoT                 | Programs inside machines and smart devices   |
| Game development                         | Mobile, computer and console games           |
| Cyber security                           | Protecting computers and data from attacks   |
| System programming                       | Operating systems, compilers, device drivers |

### 3. Popular Programming Languages Today

A **programming language** is a special language, with its own words and grammar, used to write programs. Just as people speak Assamese, Bodo, Hindi or English, programmers choose a language that suits the job. Hundreds of languages exist; these are among the most widely used:

| Language          | Mainly used for                                                                  |
|-------------------|----------------------------------------------------------------------------------|
| <b>C</b>          | Operating systems, embedded systems (programs inside machines), compilers        |
| <b>C++</b>        | Games, web browsers, high-speed software                                         |
| <b>Python</b>     | Artificial Intelligence, data science, automation; known for easy, readable code |
| <b>Java</b>       | Banking and business software, Android apps                                      |
| <b>JavaScript</b> | Interactive websites; runs inside every web browser                              |
| <b>C#</b>         | Windows software and games made with the Unity engine                            |
| <b>Kotlin</b>     | Android apps (Google's preferred language for Android)                           |
| <b>Swift</b>      | iPhone and iPad apps                                                             |
| <b>Go</b>         | Cloud servers and internet services                                              |
| <b>Rust</b>       | Fast and memory-safe system software                                             |
| <b>SQL</b>        | Storing and searching data in databases                                          |

Notice that **C**, **C++**, **Java**, **JavaScript** and **C#** all look similar. They borrowed much of their grammar from C. Learn C well, and these languages become much easier.

#### Did you know?

HTML (used for web pages) is a **markup language**, not a programming language: it describes how a page looks, but cannot make decisions or calculate. You will study it in a later semester.

## 4. Why Learn C? A Short History

### STORY · THE MOTHER OF LANGUAGES

*Arpan put his hand up. "Sir, everyone says Python is easy. Why are we starting with C?" Sir pointed to Arpan's phone. "The Android system running your game is built on the Linux kernel, and that kernel is written mostly in C. Python itself is usually run by a program written in C. If you learn C, you learn how the computer really works."*

### History of C

| Year        | Event                                                                                   |
|-------------|-----------------------------------------------------------------------------------------|
| 1967        | BCPL language developed by Martin Richards                                              |
| 1970        | Ken Thompson creates the B language at Bell Labs, USA                                   |
| 1972        | <b>Dennis Ritchie develops C at AT&amp;T Bell Laboratories</b> , as an improvement of B |
| 1973        | The UNIX operating system is rewritten in C                                             |
| 1978        | Brian Kernighan and Dennis Ritchie publish <i>The C Programming Language</i> ("K&R C")  |
| 1989        | ANSI standardizes C (ANSI C or C89); ISO adopts it in 1990 (C90)                        |
| 1999 – 2024 | New versions: C99, C11, C17 and C23                                                     |

### Why C is still important

- **Fast and efficient:** C programs run very close to the speed of the hardware.
- **Close to the machine:** C can work directly with memory (pointers, Unit 4), so it is used for operating systems and device drivers.
- **Portable:** a C program written on one computer can be compiled and run on another with few or no changes.
- **Small and simple:** C has only a small set of keywords (32 in the original standard), so it is easy to learn the core.
- **Foundation for other languages:** C++, Java, C#, JavaScript and many others follow C's style.
- **Used everywhere:** the Linux kernel, the Git version-control system, the SQLite database and the standard Python interpreter (CPython) are written in C. It is also the main language for embedded systems.

### Did you know?

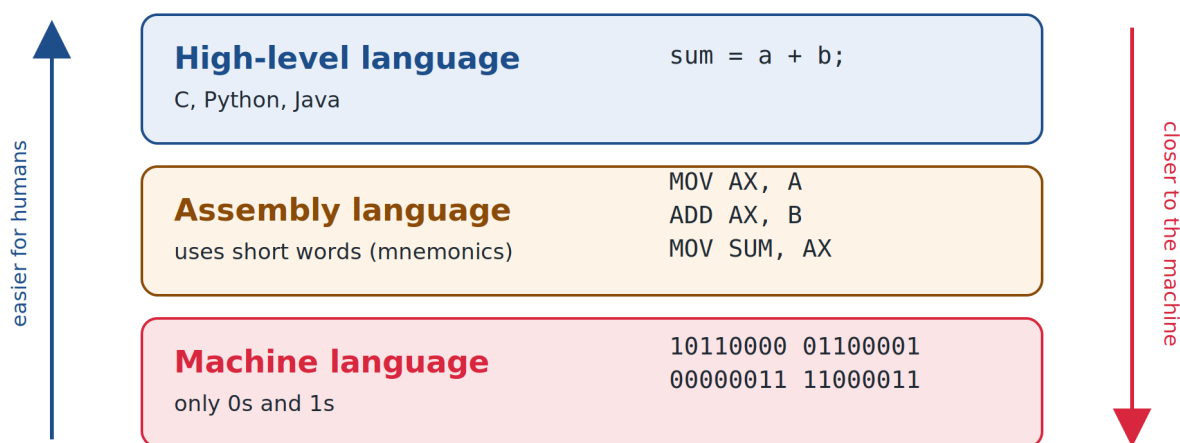
C is often called a **middle-level language**: it is a high-level language, but it can also do low-level work such as handling memory addresses and individual bits.

## Part B: Talking to the Computer

### 5. Low-Level vs High-Level Languages

#### STORY · WHY CAN'T THE COMPUTER UNDERSTAND ASSAMESE?

Sonajit asked, "Sir, why can't I just type add these two numbers in Assamese?" Sir explained that the computer's processor understands only electrical signals: ON and OFF, written as 1 and 0. Every word we type must finally be turned into these 0s and 1s.



The three levels of programming languages, from the machine's language to ours.

#### Low-level languages

- **Machine language** (first generation): instructions written only in binary (0 and 1). It is the only language the processor understands directly. It runs fastest, but is very hard to write, read and correct.
- **Assembly language** (second generation): uses short words called **mnemonics** such as **MOV**, **ADD** and **SUB** in place of binary codes. A program called an **assembler** converts it into machine language.
- Both are **machine dependent**: a program written for one type of processor will not run on another.

#### High-level languages

- Use English-like words and familiar mathematical symbols: `sum = a + b;`
- Easy to learn, write, read and correct. One line can do the work of many machine instructions.
- **Machine independent (portable)**: the same program can run on different computers after translation.
- Need a translator (a **compiler** or an **interpreter**) to convert them into machine language.
- Examples: C, C++, Java, Python, JavaScript.

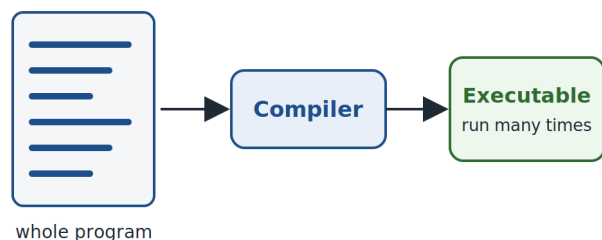
| Basis          | Low-level language                    | High-level language                 |
|----------------|---------------------------------------|-------------------------------------|
| Form           | Binary codes or mnemonics             | English-like words and symbols      |
| Understood by  | Machine (directly or almost directly) | Humans; must be translated          |
| Ease of use    | Difficult to write and correct        | Easy to write, read and correct     |
| Speed          | Very fast                             | Slightly slower (after translation) |
| Portability    | Machine dependent                     | Machine independent                 |
| Memory control | Full control of hardware              | Less direct control                 |
| Translator     | Assembler (for assembly)              | Compiler or interpreter             |
| Examples       | Machine code, assembly language       | C, C++, Java, Python                |

## 6. Compiled vs Interpreted Languages

### STORY · TWO KINDS OF TRANSLATORS

"Imagine," said Sir, "a famous Assamese novel is to be read in Japan. One way: a translator translates the whole book first, and then Japanese readers read the printed translation as many times as they like. Another way: at a meeting, an interpreter stands next to the speaker and translates each sentence as it is spoken." Kallal said, "So the first is a compiler, and the second is an interpreter!"

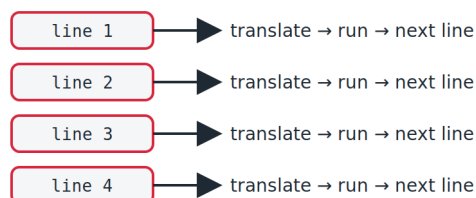
#### Compiler: translate first, run later



whole program

Example: C, C++, Go, Rust

#### Interpreter: translate while running



stops at the first line with an error

Example: Python, JavaScript

### Definitions

**Compiler:** a program that translates the **whole** source program into machine code at once, and lists all errors together. If there are no errors, it produces an **executable file** that can be run again and again without translating again.

**Interpreter:** a program that translates and runs the source program **one line (statement) at a time**. It stops at the first line with an error. The program is translated every time it runs.

**Assembler:** translates assembly language into machine language.

| Basis                     | Compiler (compiled language)   | Interpreter (interpreted language)               |
|---------------------------|--------------------------------|--------------------------------------------------|
| Translation               | Whole program at once          | Line by line                                     |
| Output file               | Creates an executable file     | No separate executable                           |
| Execution speed           | Faster                         | Slower                                           |
| Errors                    | All reported after compilation | Reported one at a time, when the line is reached |
| Translating again         | Not needed to run again        | Needed every time the program runs               |
| Source code needed to run | No (only the executable)       | Yes                                              |
| Examples                  | C, C++, Go, Rust               | Python, JavaScript, PHP                          |

**C is a compiled language.** Some languages use both methods: Java is compiled into *bytecode*, which is then run by the Java Virtual Machine.

## 7. Structure of a C Program

### STORY · EVERY LETTER HAS A FORMAT

*Sir asked, "When you write a leave application, what comes first?" Sanjita answered at once: "To, the Principal... then the date, subject, the body, and at the end, yours faithfully." "Right. A C program also has a fixed format, with sections in a fixed order."*

| No. | Section            | Purpose                                                   | Needed?                                |
|-----|--------------------|-----------------------------------------------------------|----------------------------------------|
| 1   | Documentation      | Comments: program name, author, date, purpose             | Optional (good practice)               |
| 2   | Link               | <code>#include</code> lines that link header files        | Needed when library functions are used |
| 3   | Definition         | <code>#define</code> symbolic constants                   | Optional                               |
| 4   | Global declaration | Variables and function declarations used by all functions | Optional                               |
| 5   | main() function    | Where execution starts: declarations and statements       | <b>Compulsory</b>                      |
| 6   | Sub-program        | User-defined functions (Unit 4)                           | Optional                               |

## Program 1: A C program showing all six sections

```

/* Program : Area of a circle                                1. Documentation
   Author  : Kallal Kalita, B.Sc. 1st Semester
   College : PDUAM, Amjonga                                  */

#include <stdio.h>                                           /* 2. Link section */

#define PI 3.14159                                          /* 3. Definition section */

float area(float r);                                       /* 4. Global declaration section */

int main()                                                 /* 5. main() function section */
{
    float radius = 7.0;
    printf("Area of the circle = %.2f\n", area(radius));
    return 0;
}

float area(float r)                                         /* 6. Sub-program (user function) section */
{
    return PI * r * r;
}

```

### OUTPUT

```
Area of the circle = 153.94
```

A **comment** is text between `/*` and `*/` (or after `//` to the end of the line). The compiler ignores comments; they are written to help people understand the program.

## 8. Header Files

A **header file** is a file with the extension `.h` that contains **declarations** of library functions, macros and constants. When we write `#include <stdio.h>`, the preprocessor copies the contents of `stdio.h` into our program, so the compiler knows about functions such as `printf()` and `scanf()`.

- `#include <file.h>` (angle brackets): look in the system's standard folders. Used for library header files.
- `#include "file.h"` (double quotes): look in the current folder first. Used for our own header files.
- `#include` is a **preprocessor directive**, not a C statement, so it does **not** end with a semicolon.

| Header file           | Contains                                             | Examples                                                                                    |
|-----------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>stdio.h</code>  | Standard input and output                            | <code>printf()</code> , <code>scanf()</code> , <code>getchar()</code> , <code>puts()</code> |
| <code>stdlib.h</code> | General utilities, memory                            | <code>malloc()</code> , <code>free()</code> , <code>exit()</code> , <code>rand()</code>     |
| <code>math.h</code>   | Mathematical functions                               | <code>sqrt()</code> , <code>pow()</code> , <code>sin()</code>                               |
| <code>string.h</code> | String functions                                     | <code>strlen()</code> , <code>strcpy()</code> , <code>strcmp()</code>                       |
| <code>ctype.h</code>  | Character tests                                      | <code>isdigit()</code> , <code>toupper()</code>                                             |
| <code>conio.h</code>  | Console I/O (Turbo C / Windows only, not standard C) | <code>getch()</code> , <code>clrscr()</code>                                                |

**Note for Linux users**

`conio.h` is not part of standard C and is not available with GCC on Linux. The programs in this book use only standard header files, so they run everywhere. On Linux, the math library is linked with `-lm`: `gcc prog.c -o prog -lm`

## 9. The `main()` Function and Your First Program

**STORY · HELLO, WORLD!**

*Sir opened a terminal on the projector and typed five lines. He compiled, ran, and the screen said Hello, World! The class clapped. Sonajit said, "That's it? That's programming?" "That's the beginning," Sir said. "Hello, World has been the traditional first program ever since the famous K&R book of 1978."*

**Program 2: Hello, World!**

```
#include <stdio.h>

int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

**OUTPUT**

Hello, World!

| Line                                    | Meaning                                                                                                                             |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>#include &lt;stdio.h&gt;</code>   | Include the standard I/O header file, which declares <code>printf()</code> .                                                        |
| <code>int main()</code>                 | Every C program starts executing from <code>main()</code> . <code>int</code> means main returns an integer to the operating system. |
| <code>{</code>                          | Start of the body of main.                                                                                                          |
| <code>printf("Hello, World!\n");</code> | Print the text on the screen. <code>\n</code> moves the cursor to a new line. The statement ends with <code>;</code> .              |
| <code>return 0;</code>                  | End main and tell the operating system that the program finished successfully (0 = success).                                        |
| <code>}</code>                          | End of the body of main.                                                                                                            |

**Rules about `main()`**

A program must have **exactly one** `main()` function. Execution always starts there, wherever it is written.

C is **case sensitive**: `main`, `Main` and `MAIN` are three different names. Only `main` works.

Use `int main()` with `return 0;`. Some old books use `void main()`, which is not standard C.

### Program 3: A welcome banner

```
#include <stdio.h>

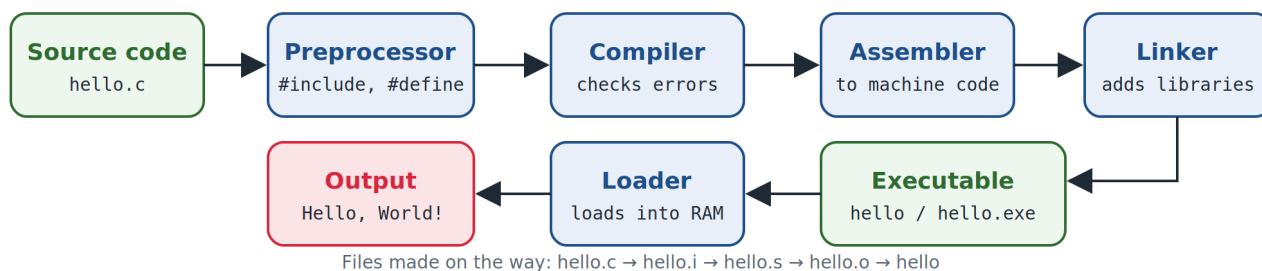
int main()
{
    printf("*****\n");
    printf("*   Welcome to the world of C!       *\n");
    printf("*   Department of Computer Science   *\n");
    printf("*   PDUAM, Amjonga                   *\n");
    printf("*****\n");
    return 0;
}
```

#### OUTPUT

```
*****
*   Welcome to the world of C!       *
*   Department of Computer Science   *
*   PDUAM, Amjonga                   *
*****
```

## 10. Compiling and Executing a Program

A computer cannot run `hello.c` directly. It must go through several steps:



**On Linux, one command does steps 1-4:**

`gcc hello.c -o hello` and `./hello` runs the program (the loader does its job).

*From source code to output: what happens when you compile and run a C program.*

| Step                   | What happens                                                                                                                                |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Editing             | The program is typed in an editor (gedit, nano, VS Code, Code::Blocks) and saved with extension <code>.c</code> .                           |
| 2. Preprocessing       | Lines beginning with <code>#</code> are handled: header files are copied in, <code>#define</code> names are replaced, comments are removed. |
| 3. Compiling           | The compiler checks the grammar (syntax) and translates the program. If there are errors, it lists them and stops.                          |
| 4. Assembling          | The assembly code is converted into machine code (an <b>object file</b> , <code>.o</code> ).                                                |
| 5. Linking             | The linker joins the object file with library code (for example, the code of <code>printf()</code> ) to make an <b>executable file</b> .    |
| 6. Loading and running | The loader puts the executable into memory (RAM), and the processor runs it.                                                                |

## On Linux (recommended in the syllabus)

### TERMINAL

```
gcc hello.c -o hello      # compile: creates the executable 'hello'
./hello                  # run it

gcc hello.c               # without -o, the executable is named a.out
./a.out
```

In Turbo C or Code::Blocks on Windows, the same steps happen when you press **Compile** and **Run** (in Turbo C: Alt+F9 to compile, Ctrl+F9 to run).

## Types of errors

### STORY · AN ERROR ON PURPOSE

*To show the class what an error looks like, Sir asked Sonajit to remove the semicolon after `printf(...)` and compile again. The compiler printed: error: expected ';' before 'return'. "Every programmer sees messages like this every day," Sir said. Sanjita noticed something useful: "It even tells us the line number."*

| Error                                 | When found                               | Example                                                                                                   |
|---------------------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Syntax error</b><br>(compile-time) | By the compiler, before the program runs | Missing <code>;</code> , misspelt <code>pritrnf</code> , unmatched <code>{</code>                         |
| <b>Linker error</b>                   | When linking                             | Calling a function that does not exist anywhere                                                           |
| <b>Run-time error</b>                 | While the program runs                   | Dividing an integer by zero                                                                               |
| <b>Logical error</b>                  | Only by checking the output              | Writing <code>a - b</code> where <code>a + b</code> was needed; the program runs but gives a wrong answer |

FROM THE FULL BOOK

## Unit 3 · Arrays and Strings

*A sample of the story style and the "Just for fun" boxes*

---

## Prologue: Sixty Variables

**a[i][j]**

```
char s[] =
"PDUAM";
```

MANY VALUES, ONE NAME

|     | [0] | [1] | [2] | [3] | [4] | [5] |
|-----|-----|-----|-----|-----|-----|-----|
| [0] | 69  | 88  | 57  | 44  | 27  | 33  |
| [1] | 96  | 10  | 53  | 74  | 69  | 87  |
| [2] | 20  | 52  | 80  | 88  | 99  | 15  |
| [3] | 58  | 31  | 67  | 64  | 30  | 31  |
| [4] | 40  | 16  | 24  | 26  | 74  | 85  |

### STORY · THE PROBLEM OF SIXTY STUDENTS

*At the end of Unit 2, Sonajit had asked a question: "To store the marks of sixty students, do we need sixty variables?" Sir decided to try it on the board.*

```
int a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x,
y, z,
```

*He stopped. "After z, what? aa, ab, ac...? By the time I finish declaring them, you will all have passed out of college." The class laughed. "And then," he went on, "to find the total, I would write a + b + c + ... sixty times. And next year, if seventy students join, I would rewrite the whole program!"*

*Then he wrote a single line: `int marks[60];` "One name. Sixty boxes. That is an **array**."*

# Part A: One-Dimensional Arrays

## 1. Why Arrays?

An ordinary variable holds only one value at a time. When a program must store many values of the same kind (marks of a class, rainfall of a month, prices of items), using separate variables is hectic and time-consuming. The array was introduced to solve this problem: a single unit that can hold all those values.

### Definition

An **array** is a collection of a fixed number of elements of the **same data type**, stored in **continuous memory locations** and referred to by a **single name**. Each element is accessed by its position number, called the **index** (or subscript).

### Features of an array

- All elements have the same data type: an `int` array holds only integers.
- Elements are stored next to each other in memory.
- The size is fixed when the array is declared.
- The index of the first element is **0** and of the last is **size – 1**.
- With a loop, thousands of elements can be processed with just a few lines of code.

Arrays are of three kinds: **one-dimensional** (a list), **two-dimensional** (a table or matrix) and **multidimensional** (three or more dimensions).

### JUST FOR FUN

Shopkeeper: "How many eggs do you want?" Programmer: "Give me `eggs[12]`."  
Shopkeeper hands over a tray. Programmer: "Perfect, an array!"

An egg tray, a row of hostel rooms, the coaches of a train: each is a set of similar things, numbered, kept side by side.

## 2. Declaring an Array

### SYNTAX

```
data_type array_name[size];

int   marks[60];      /* 60 integers */
float rain[31];       /* 31 real numbers */
char  name[30];       /* 30 characters */
```

- **data\_type**: the type of every element (`int`, `float`, `char` ...).
- **array\_name**: any valid identifier.
- **size**: the maximum number of elements the array can hold. In standard C it is written as a positive integer constant: `int a[n]`; with a variable `n` is allowed only from C99 onwards, and not in Turbo C.
- Memory used = size × size of one element. On GCC (Linux), `int` takes 4 bytes, so `int marks[60]` takes 240 continuous bytes. (In old Turbo C, `int` was 2 bytes, so it would take 120 bytes.)

```
int marks[5] = {78, 65, 92, 54, 81};
```

| name    | marks[0] | marks[1] | marks[2] | marks[3] | marks[4] |
|---------|----------|----------|----------|----------|----------|
| value   | 78       | 65       | 92       | 54       | 81       |
| address | 1000     | 1004     | 1008     | 1012     | 1016     |

5 elements × 4 bytes = 20 continuous bytes; the index starts at 0 and ends at size – 1

*An array in memory: one name, five boxes, continuous addresses.*

The name of the array (**marks**) stands for the address of its first element. This is why, in Unit 4, an array name behaves like a pointer.

### 3. Initializing an Array

Values can be given to an array when it is declared, inside braces:

#### COMPILE-TIME INITIALIZATION

```
int a[5] = {4, 9, 8, 6, 1};    /* a[0]=4, a[1]=9, a[2]=8, a[3]=6, a[4]=1 */
int a[] = {4, 9, 8, 6, 1};    /* size is counted automatically: 5 */
int b[5] = {4, 7};           /* b[0]=4, b[1]=7, the rest are 0 */
int z[5] = {0};              /* all elements are 0 */
```

- If the list has fewer values than the size, the remaining elements become **zero**.
- Giving more values than the size is an error: `int a[3] = {1, 2, 3, 4};`
- An array that is declared but not initialized (inside a function) contains **garbage values**.
- One array cannot be assigned to another: `b = a;` is an error. Copy element by element with a loop.
- Values can also be given while the program runs (**run-time initialization**), usually with `scanf()` in a loop.

## Program 1: Ways of initializing arrays

```
#include <stdio.h>

int main()
{
    int a[5] = {4, 9, 8, 6, 1};           /* full initialization */
    int b[5] = {4, 7};                   /* partial: rest become 0 */
    int c[] = {10, 20, 30};               /* size taken from the list (3) */
    int z[5] = {0};                       /* all elements 0 */
    int i, n;

    printf("a : ");
    for (i = 0; i < 5; i++) printf("%d ", a[i]);
    printf("\nb : ");
    for (i = 0; i < 5; i++) printf("%d ", b[i]);
    printf("\nz : ");
    for (i = 0; i < 5; i++) printf("%d ", z[i]);

    n = sizeof(c) / sizeof(c[0]);         /* number of elements */
    printf("nc has %d elements and takes %d bytes\n", n, (int)sizeof(c));
    return 0;
}
```

### OUTPUT

```
a : 4 9 8 6 1
b : 4 7 0 0 0
z : 0 0 0 0 0
c has 3 elements and takes 12 bytes
```

`sizeof(c) / sizeof(c[0])` is a useful trick to find the number of elements of an array.

## 4. Accessing Array Elements

An element is used by writing the array name followed by its index in square brackets: `marks[0]`, `marks[i]`, `marks[i+1]`. An element can be used anywhere an ordinary variable can: in `scanf()`, `printf()`, expressions and assignments.

```
marks[2] = 92;           /* store */
printf("%d", marks[2]);  /* print */
total = marks[0] + marks[1]; /* use in an expression */
scanf("%d", &marks[3]);  /* read: & is still needed */
```

### JUST FOR FUN

Why do C programmers start counting from zero?

Because the index tells *how far* an element is from the start. The first element is zero steps away, so it is `a[0]`. (That is also why a programmer's first birthday party is jokingly called the *zeroth* birthday.)

### STORY · RAIN, RAIN, GO AWAY

*It had rained for a whole week in Goalpara. Sanjita suggested recording the daily rainfall in an array and letting the program find the wettest day.*

## Program 2: A week of rainfall

```
#include <stdio.h>

int main()
{
    float rain[7], total = 0;
    int i, maxDay = 0;

    printf("Enter rainfall (mm) in Goalpara for 7 days:\n");
    for (i = 0; i < 7; i++)
    {
        printf("Day %d: ", i + 1);
        scanf("%f", &rain[i]);
        total = total + rain[i];
        if (rain[i] > rain[maxDay])
            maxDay = i;
    }

    printf("\nDay : ");
    for (i = 0; i < 7; i++)
        printf("%6d", i + 1);
    printf("\nmm : ");
    for (i = 0; i < 7; i++)
        printf("%6.1f", rain[i]);

    printf("\n\nTotal rainfall = %.1f mm\n", total);
    printf("Average per day = %.1f mm\n", total / 7);
    printf("Wettest day = Day %d (%.1f mm). Don't forget your umbrella!\n",
           maxDay + 1, rain[maxDay]);
    return 0;
}
```

### OUTPUT

```
Enter rainfall (mm) in Goalpara for 7 days:
Day 1: 12.5
Day 2: 0
Day 3: 34.2
Day 4: 8
Day 5: 56.8
Day 6: 21
Day 7: 3.5

Day :      1      2      3      4      5      6      7
mm :   12.5    0.0   34.2    8.0   56.8   21.0    3.5

Total rainfall = 136.0 mm
Average per day = 19.4 mm
Wettest day = Day 5 (56.8 mm). Don't forget your umbrella!
```

The loop runs from `i = 0` to `i < 7`, but prints `i + 1` so that people see Day 1 to Day 7.

### Out of bounds: C will not stop you

In `int a[5];` the valid indices are 0 to 4. C does **not** check the index. Writing `a[5] = 10;` or `a[100] = 10;` compiles, but writes into memory that belongs to something else. The result is unpredictable: wrong values, or a crash.

*It is like a 60-seat college bus: C will happily give you seat number 61, and you will end up sitting on someone's lap.*

**FROM THE FULL BOOK**

# **Solved Practicals**

*Two sample pages from the 30 solved syllabus practicals*

---

### About these solutions

The syllabus list has 34 items, but item 1 is a note and items 18–20 are parts of item 17, so there are 30 programs. They are numbered 1–30 here, with the syllabus number shown for each.

The code is kept short and simple, using only what is taught in the syllabus. Every program was compiled with GCC and run on Linux (the environment advised in the syllabus); the outputs shown are from those runs, with the user's input shown after each prompt.

All programs use `int main()` with `return 0;` and avoid `getch()` and `<conio.h>`, which are not available on Linux.

Compile and run on Linux: `gcc prog.c -o prog` then `./prog`

## Unit 1: Getting Started with C

### Practical 1: Print "Hello World" (Syllabus Q.2)

```
#include <stdio.h>

int main()
{
    printf("Hello World\n");
    return 0;
}
```

#### OUTPUT

```
Hello World
```

### Practical 2: Sum, product and difference of two numbers (Syllabus Q.3)

```
#include <stdio.h>

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Sum          = %d\n", a + b);
    printf("Product       = %d\n", a * b);
    printf("Difference = %d\n", a - b);
    return 0;
}
```

#### OUTPUT

```
Enter two numbers: 12 5
Sum          = 17
Product       = 60
Difference = 7
```

## Unit 2: Control Structures

### Practical 3: Greatest and smallest of three numbers (Syllabus Q.4)

```
#include <stdio.h>

int main()
{
    int a, b, c, big, small;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    big = a;
    if (b > big) big = b;
    if (c > big) big = c;

    small = a;
    if (b < small) small = b;
    if (c < small) small = c;

    printf("Greatest = %d\n", big);
    printf("Smallest = %d\n", small);
    return 0;
}
```

#### OUTPUT

```
Enter three numbers: 25 8 40
Greatest = 40
Smallest = 8
```

FROM THE FULL BOOK

# Appendix · The Examination

*Exam pattern, time plan and how much to write*

---

## Appendix A: The Examination

### A.1 Pattern of the question paper

The end-semester theory paper of Introduction to C-Programming carries **45 marks** and is of **2 hours**. The model paper in Appendix D follows the current marking pattern:

| Question | Type                                      | Choice            | Marks              |
|----------|-------------------------------------------|-------------------|--------------------|
| 1        | Objective: true/false, fill in the blanks | All compulsory    | $1 \times 4 = 4$   |
| 2        | Very short answers                        | Any three of four | $2 \times 3 = 6$   |
| 3        | Short answers                             | Any three of five | $5 \times 3 = 15$  |
| 4        | Long answers (may have two parts)         | Any two of three  | $10 \times 2 = 20$ |
|          | <b>Total</b>                              |                   | <b>45</b>          |

The 2025 paper (Appendix B) used an earlier distribution of the same 45 marks:  $1 \times 5$ ,  $2 \times 5$ ,  $5 \times 4$  and  $10 \times 1$ . The kinds of questions are the same, so it is still excellent practice.

#### Check your year's pattern

Patterns can change. Always confirm the pattern for your examination year from the university notification or your teachers.

### A.2 Planning the two hours

With 120 minutes for 45 marks, a good rule is about **2.5 minutes per mark**, keeping 10 minutes to read the paper and 5 minutes to check at the end.

| Step                  | Time   | What to do                                          |
|-----------------------|--------|-----------------------------------------------------|
| Read the paper        | 10 min | Read every question; tick the ones you will attempt |
| Question 1 (4 marks)  | 5 min  | Quick answers; do not leave any blank               |
| Question 2 (6 marks)  | 15 min | Short, exact answers                                |
| Question 3 (15 marks) | 35 min | About 12 minutes per answer                         |
| Question 4 (20 marks) | 50 min | About 25 minutes per answer                         |
| Check                 | 5 min  | Question numbers, semicolons, missing parts         |

### A.3 How much to write

| Marks | What a good answer contains                                                                                           |
|-------|-----------------------------------------------------------------------------------------------------------------------|
| 1     | Only the answer: True / False or the missing word                                                                     |
| 2     | A definition in one or two sentences, plus one example or a two-point difference                                      |
| 5     | Definition, explanation in points, syntax, a small example or a short program, and its output                         |
| 10    | Introduction, full explanation, diagram or table, a complete program with comments and output, and a short conclusion |

- Write programs neatly with `#include`, `int main()`, `return 0;` and comments. Show a sample output.
- Use tables for "differences" questions; examiners can mark them quickly.
- Draw a diagram where it helps: memory boxes for pointers, a flowchart for loops, a matrix grid for 2-D arrays.
- Underline key terms. Start each answer on a fresh line with its question number.

## A.4 Frequently asked topics

These topics appear in both papers in this appendix, so prepare them well:

| Topic                                                                                                         | Unit |
|---------------------------------------------------------------------------------------------------------------|------|
| C tokens, keywords, variables, operators                                                                      | 1    |
| break and continue; loops                                                                                     | 2    |
| Matrix programs (transpose, addition, multiplication)                                                         | 3    |
| Call by value vs call by reference; swap with pointers                                                        | 4    |
| Pointers and their operations                                                                                 | 4    |
| Recursion, prime and factorial functions                                                                      | 4    |
| Structure vs union; Student structure program                                                                 | 5    |
| File functions: <code>fopen()</code> , <code>fclose()</code> , <code>fprintf()</code> , <code>fscanf()</code> | 6    |

**END OF THE FREE PREVIEW**

# Get the complete book

Introduction to C-Programming · 235 pages · First Edition 2026

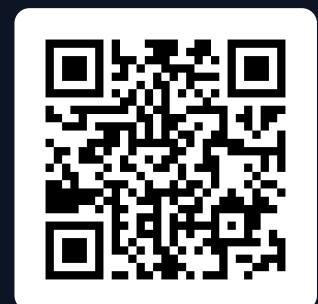
- All 6 units of the FYUGP syllabus, in easy story style
- 130+ programs, every one tested with GCC on Linux
- 30 solved syllabus practicals
- Previous year question paper (2025) with full solutions
- Model question paper on the new examination pattern
- Exam questions, viva questions and practice programs in every unit

## Price: ₹30 only (e-book, PDF)

1. Fill the order form: [forms.gle/CET7Je3Td9eCWjyp9](https://forms.gle/CET7Je3Td9eCWjyp9)
2. Pay ₹30 by UPI and enter the transaction ID in the form
3. Receive your personal copy by e-mail or WhatsApp

Also coming soon on Google Play Books.

Queries: sisirrajbongshi@gmail.com



**Scan to order**

ISBN 978-93-345-4161-8

## ABOUT THIS BOOK

# Learn C the way you read a story.

Written unit by unit for the new FYUGP syllabus of Gauhati University, this book takes first-semester students from their very first program to files, pointers and structures. Every topic opens with a short classroom story, every idea is explained in simple language, and every one of its 130+ programs was compiled and run with GCC on Linux.

### UNIT 1

Getting Started with C

### UNIT 2

Control Structures

### UNIT 3

Arrays and Strings

### UNIT 4

Functions and Pointers

### UNIT 5

Structures and Unions

### UNIT 6

File Processing in C

Plus 30 solved syllabus practicals, a previous year question paper with full solutions, and a model question paper on the new examination pattern.

## ABOUT THE AUTHOR



### Dr. Sisir Kumar Rajbongshi

Dr. Sisir Kumar Rajbongshi is an Assistant Professor in the Department of Computer Science at Pandit Deendayal Upadhyaya Adarsha Mahavidyalaya (PDUAM), Amjonga, Goalpara, Assam. He holds a Ph.D. from Gauhati University and has a keen interest in both theoretical and applied aspects of Computer Science. His major research interests include Fuzzy Sets and Fuzzy Algebra, Artificial Intelligence, Machine Learning, Data Science, Federated Learning, and intelligent computational applications. He has published research papers and contributed to books and book chapters in diverse areas of Computer Science.

Alongside his research pursuits, he is actively engaged in teaching, academic administration, undergraduate research, and mentoring students. He is particularly interested in connecting computational techniques with real-world problems and encouraging young learners to develop innovative and research-oriented solutions. Through his academic and professional activities, he continues to contribute to the advancement of Computer Science education and research in Assam.

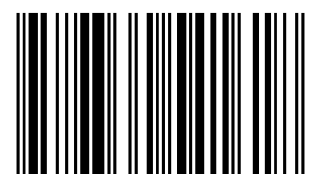
## Introduction to C-Programming

**B.Sc. 1st Semester • Computer Science • Major & Minor**

As per the FYUGP syllabus of Gauhati University under NEP 2020

First Edition 2026 • COM0100104MJ • COM0101004MN

ISBN 978-93-345-4161-8



9789334541618